

DS CW2, text sentiment, spatial

Question 1

Generative AI tools have gained a reputation for their ability to produce texts in the style of famous authors. You are asked to put this claim to the test using the work by Charles Dickens.

The file “Dickens_Fakes.csv” contains five pieces of text written by ChatGPT in the style of Charles Dickens, and “Dickens_Originals.csv” provides ten randomly selected chapters from the books “A Tale of Two Cities” and “Great Expectations”. Explore the similarities and differences between the original and artificially produced pieces of text. Your analysis should consider both the words used and the emotional intent of the texts.

What do you conclude regarding ChatGPT’s ability to produce work in the style of Charles Dickens? How confident should we be about your conclusion?

Answer

Load in the libraries

```
library( dplyr )
library( ggplot2 )
library( tidytext )
library( wordcloud )
library( stringr )
library(tidyr)
library(patchwork)
```

It would first be useful to make a comparison of word frequencies in each text.

Load the data

```
Dickens_original_raw <- read.csv("Dickens_Originals.csv")
Dickens_fake_raw <- read.csv("Dickens_Fakes.csv")
```

Clean the data in both texts

```
data(stop_words)
DickensOG <- Dickens_original_raw %>%
  unnest_tokens(word, Text) %>%
  mutate(word = gsub("_", "", word))

DickensFake <- Dickens_fake_raw %>%
  unnest_tokens(word, Text) %>%
  mutate(word = gsub("_", "", word))
```

Calculate the word frequencies in each text

```
DickensOG_Count <- DickensOG %>%
  count(word, sort = TRUE) %>%
  mutate(`term frequency` = n / sum(n), rank = row_number()) %>%
  anti_join(stop_words, by = join_by(word))

DickensFake_Count <- DickensFake %>%
  count(word, sort = TRUE) %>%
  mutate(`term frequency` = n / sum(n), rank = row_number()) %>%
  anti_join(stop_words, by = join_by(word))
```

Now we combine the data sets together

```
frequency_combined <- full_join(DickensOG_Count, DickensFake_Count, by = "word") %>%
  rename(DickensOG = `term frequency.x`, DickensFake = `term frequency.y`)
```

Before analysing the frequency terms, we account for words that only appear in one text, hence we replace any NA with 0.

```
frequency_combined <- frequency_combined %>%
  mutate( DickensOG=case_when(is.na(DickensOG) == TRUE ~ 0, .default=DickensOG),
  DickensFake= case_when(is.na(DickensFake) == TRUE ~ 0, .default= DickensFake))
```

Plot the data

```
ggplot( frequency_combined, aes( x=DickensOG, y=DickensFake)) +
  geom_point()+
  geom_text( aes(label=word), check_overlap= TRUE, vjust=1.5) +
  xlim( c(0,0.003)) + ylim(c(0,0.003)) +
  coord_trans( x="sqrt", y="sqrt") + theme_bw() +
  labs( x="Term Frequency in Original Books",
  y="Term Frequency in AI Texts" )
```



From this plot we can see there are a few words of similar frequency in the AI texts and the original text, with the most similar frequencies including words such as : ‘time’, ‘air’ and ‘hand’, indicating that AI is able to produce some words of the same frequency, however we observe that there is not a huge cluster of words around what would be the line $y = x$, indicating that AI does not produce a lot of the same frequency of words, and is more diverse in its vocabulary use, showing that AI is not producing a lot of the same word patterns as Dickens. High frequency words in the texts include names of characters, showing that AI has picked up on this pattern of Dickens’ work.

We find the 10 most common words in the AI and original works. We create a bar plot for the raw frequency of the original works in descending frequency order

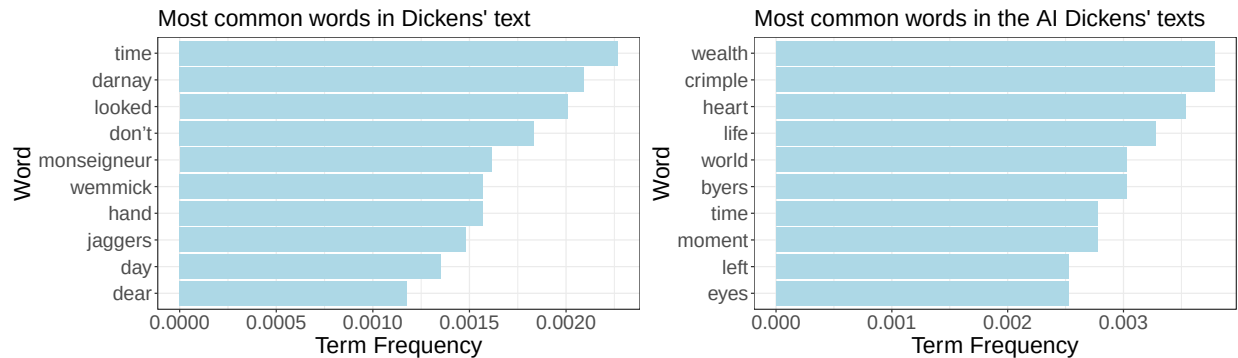
```
Real_words <- DickensOG_Count %>%
  arrange(desc(`term frequency`)) %>%
  slice_head(n = 10) %>%
  mutate(word = reorder(word, `term frequency`)) %>%
  ggplot(aes(x = `term frequency`, y = word)) + geom_col(fill = "lightblue") +
  labs(x = "Term Frequency", y = "Word", title = "Most common words in Dickens' text") +
  theme_bw() + theme(axis.title = element_text(size = 20),
    axis.text = element_text(size = 17),
    plot.title = element_text(size = 20))
```

We next create a bar plot of the raw frequency in the AI works in descending frequency order

```
Fake_words <- DickensFake_Count %>%
  arrange(desc(`term frequency`)) %>%
  slice_head(n = 10) %>%
```

```
mutate(word = reorder(word, `term frequency`)) %>%
ggplot(aes(x = `term frequency`, y = word)) + geom_col(fill = "lightblue") +
labs(x = "Term Frequency", y = "Word",
     title = "Most common words in the AI Dickens' texts") +
theme_bw() + theme(axis.title = element_text(size = 20),
                   axis.text = element_text(size = 17),
                   plot.title = element_text(size = 20))
```

Real_words + Fake_words



The bar plot above does not include stop words. It tells us that apart from the word 'time' appearing frequently in both files, the most frequent word (0.0022) in the actual Dickens work and the 6th most common word in the AI generated work (0.0027), there is no overlap in the remaining top 10 words. This suggests that while AI may be aware of recurring themes in Dickens' work, it struggles to replicate the most frequent vocabulary that is specific to Dickens' work.

Now we analyse the Term frequency-inverse document frequency to assess important words in the original and AI works. Tf-IDF for the original works. We remove stop words, and rearrange the tf-idf frequency in descending order, presenting the top 6 words.

```
DickensOG_Tf <- DickensOG %>%
  anti_join(stop_words) %>%
  count( Title, word, sort=TRUE )

DickensOG_tf.idf <- DickensOG_Tf %>%
  bind_tf_idf( word, Title, n ) %>%
  arrange( desc(tf_idf) ) %>%
  mutate( idf=round(idf,2) )
  head( DickensOG_tf.idf )
```

##	Title	word	n	tf	idf	tf_idf
## 1	A Tale of Two Cities	darnay	48	0.012772751	0.69	0.008853397
## 2	Great Expectations	wemmick	36	0.009928296	0.69	0.006881770
## 3	A Tale of Two Cities	monseigneur	37	0.009845663	0.69	0.006824493
## 4	Great Expectations	jaggers	34	0.009376724	0.69	0.006499450
## 5	Great Expectations	joe	25	0.006894650	0.69	0.004779007
## 6	A Tale of Two Cities	charles	24	0.006386376	0.69	0.004426698

We do the same for the AI generated work

```
DickensFake_Tf <- DickensFake %>%
  anti_join(stop_words) %>%
  count( Title, word, sort=TRUE )

DickensFake_tf.idf <- DickensFake_Tf %>%
  bind_tf_idf( word, Title, n ) %>%
  arrange( desc(tf_idf) ) %>%
  mutate( idf=round(idf,2) )
  head( DickensFake_tf.idf )
```

##	Title	word	n	tf	idf	tf_idf
## 1	The Fallen Gentleman	geoffrey	7	0.04430380	1.61	0.07130421
## 2	The Downtrodden Poet	byers	12	0.03986711	1.61	0.06416364
## 3	The Miser's Toll	crimble	15	0.03712871	1.61	0.05975636
## 4	The Shopkeeper's Lament	ambrose	5	0.02873563	1.61	0.04624822
## 5	The Fallen Gentleman	society	4	0.02531646	1.61	0.04074526
## 6	The Orphan's Heart	mary	10	0.02358491	1.61	0.03795844

In the original works, frequent character names such as Darnay, Wemmick, and Monseigneur appear in the TF-IDF analysis with relatively high values, suggesting that these characters are significant to the books. We can see that AI has mimicked Dickens' theme of repeatedly using characters' names (to similar frequencies), and has used more 19th century style names, mimicking the time period of his stories.

It is now useful to perform a sentiment analysis to see how AI compares sentimentally.

It would be better to compare the mean AFINN scores rather than the cumulative as we do not want to assess how AFINN score changes throughout the books, but are instead looking at random samples of texts from the books . Load in the data

```
AFINN <- read.csv( "AFINN Sentiment Lexicon.csv" )
```

We create an average sentiment bar plot for the AI generated texts. We first need to assign numbers to the rows, and then join the data with AFINN to find their sentiments.

```
Fakesentiment <- DickensFake %>%
  mutate(line = row_number())
```

```
DickensFake_AFINN <- inner_join(Fakesentiment, AFINN)
```

We can now calculate the average word sentiment score in each text. Then we plot the data

```
Fake_AFINNplot <- DickensFake_AFINN %>%
  group_by(Title) %>%
  summarise(sentiment = mean(value)) %>%
  ggplot(aes(x = Title, y = sentiment)) + geom_col(fill = "lightblue") + theme_bw() +
  labs(x = "Title of work", y = "Sentiment score", title = "Average AI sentiment score") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

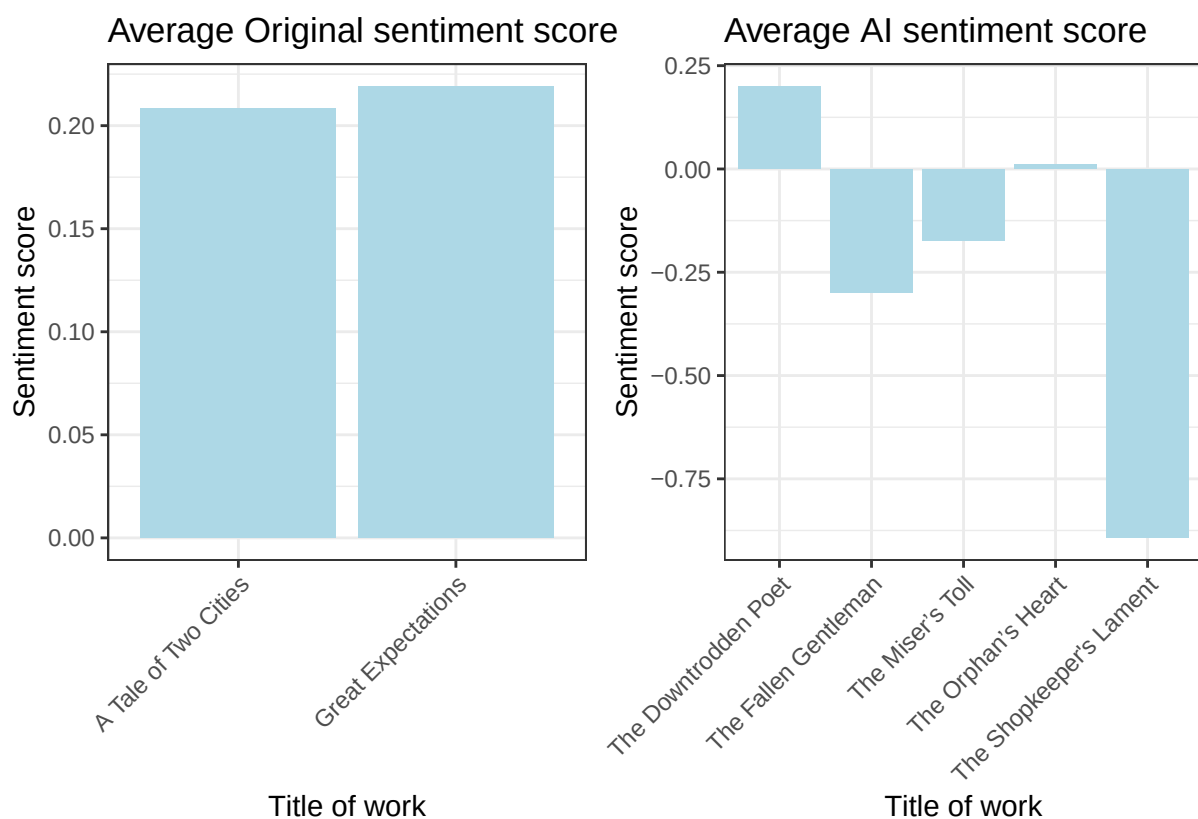
We now do the same and create an average sentiment bar plot for the Dickens' work

```
Realsentiment <- DickensOG %>%
  mutate(line = row_number())
```

```
DickensOG_AFINN <- inner_join(Realsentiment, AFINN)
```

```
OG_AFINNplot <- DickensOG_AFINN %>%
  group_by(Title) %>%
  summarise(sentiment = mean(value)) %>%
  ggplot(aes(x = Title, y = sentiment)) + geom_col(fill = "lightblue") + theme_bw() +
  labs(x = "Title of work", y = "Sentiment score", title = "Average Original sentiment score") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

```
OG_AFINNplot + Fake_AFINNplot
```



A comparison of the AFINN sentiment scores tell us that in Dickens work, he uses slightly positive language, with average sentiment scores very similar in both books (around 0.22), while in the AI generated work we can see that there is more variation in the average sentiment score (-1 to 0.25) with more negative texts. AI has managed to produce fairly neutral work, same as the original Dickens' works.

Overall, drawing a comparison of the sentiments is still difficult, as while we can assume that the sentiment scores for the 2 original Dickens book are fairly accurate (seeing that the 10 chapter were randomly selected), it is difficult to draw overall conclusions about the sentiment of Dickens' writing from only 2 of his books. There could be a possibility that the two selected books are unusually positive. Also the AI generated work and original works are for different texts, further complicating a direct comparison.

From the analysis made from the data given, while ChatGPT has managed to replicate some aspects of Charles Dickens' style, such as the use of old-fashioned character names and maintaining a generally neutral

sentiment, it does not fully capture the depth and consistency of Dickens' writing. Therefore while ChatGPT can mimic the surface-level features of Dickens' work, it does not replicate it in full detail.

Another point to consider is the confidence in our conclusion. The analysis is based on only two original Dickens works and five AI-generated texts. While the 10 chapters from *A Tale of Two Cities* and *Great Expectations* were randomly selected, there is still a possibility that these particular books might not represent Dickens' overall style. Furthermore, the AI-generated works are 5 random texts, and not an imitation of the same 2 original works we were given, hence differences in length and content could account for some of the variations observed. Overall, we are not too confident in our conclusion due to these aspects that may have influenced it.

Question 2

The region of Tesremos is one of Utopia's key sources for freshwater. Unfortunately, Tesremos has seen lower than expected rainfall over the past years, which in the longer term may threaten the country's capacity to deliver clean water to its citizens. Due to its importance, the government of Utopia has been monitoring ground water levels at several sites across Tesremos for decades.

The government of Utopia has now approached you to analyse some of last year's data to get a better understanding of the current situation. Specifically, they provided you with daily observations from their 102 groundwater monitoring sites across Tesremos for March and August 2024, which correspond to the beginning and end of Utopia's dry season respectively. Instead of the actual values, the government of Utopia has decided to provide you with the differences (in percent) to historical averages. As such, a value of x for a day and site in 2024 corresponds to ground water levels being $x\%$ lower (for x negative), or $x\%$ higher (for x positive), than the groundwater levels which were observed historically for that day and site.

You are also provided with a shapefile for Tesremos, and the locations of the 102 monitoring sites. To hide Tesremos's location, constants have been added to the latitude and longitude coordinates, but the shapes they define are correct.

There are two tasks the government of Utopia asks you to complete using the provided data:

- i. *Explore the spatial pattern/structure and dependence of the monthly average deficit in groundwater levels for the 102 monitoring sites separately for March and August. Discuss all the assumptions you make for the analysis.* Load in the libraries

```
library(sf)
library(lubridate)
library(sp)
library(prettymapr)
library(ggspatial)
library(gstat)
library(spatstat)
```

Load the data

```
Tesremos_Locations <- read.csv("Tesremos_Locations.csv")
Tesremos_Water <- read.csv("Tesremos_Water.csv")
TesremosShape <- read_sf("Tesremos.shp")
```

A visual approach would be to plot the monthly average deficit of the 102 sites across Tesremos on its map.

Changing the date format into something we can work with

```
Tesremos_Water <- Tesremos_Water %>%
  mutate(Date = ymd(Date), month = month(Date))
```

Working out the monthly average for each site, and changing the format of the data from rows to columns so it is easier to work with. We also store the monthly average for March and August in 2 separate data sets.

```
MarchWaterAverage <- Tesremos_Water %>%
  filter(month == 3) %>%
  summarise(across(is.numeric, mean, na.rm = TRUE)) %>%
  pivot_longer(cols = starts_with("Site_"), names_to = "ID", values_to = "Deficit")

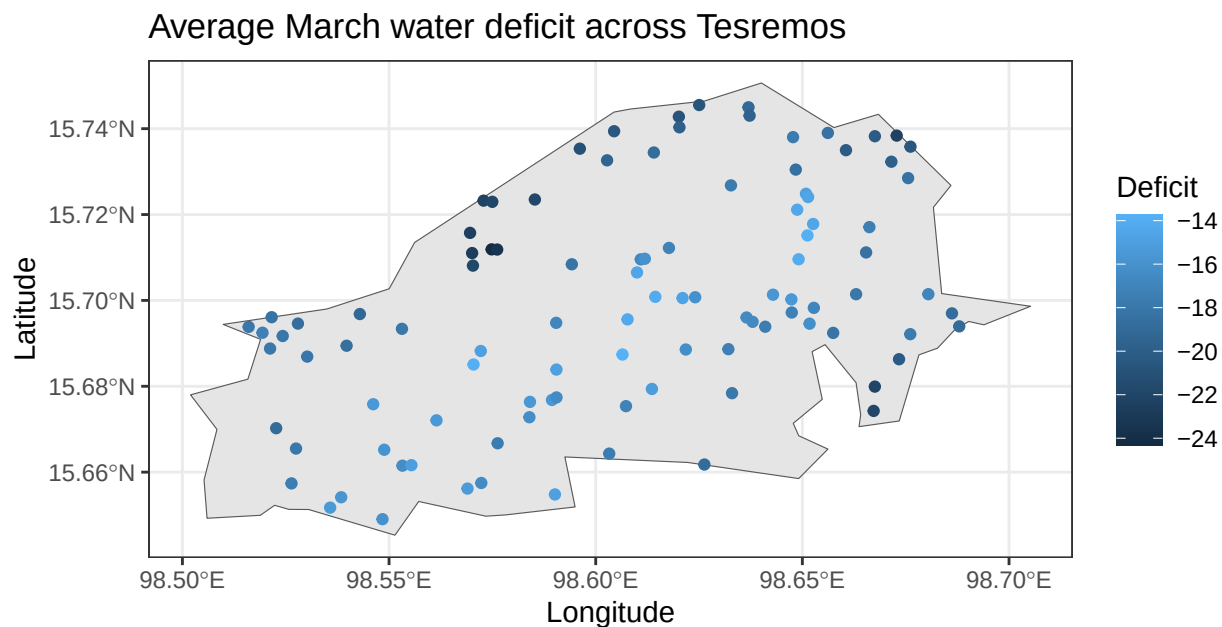
AugustWaterAverage <- Tesremos_Water %>%
  filter(month == 8) %>%
  summarise(across(is.numeric, mean, na.rm = TRUE)) %>%
  pivot_longer(cols = starts_with("Site_"), names_to = "ID", values_to = "Deficit" )
```

We merge the deficits for the sites with their locations

```
AugustWater_Merged <- AugustWaterAverage %>%
  left_join(Tesremos_Locations, by = "ID")
MarchWater_Merged <- MarchWaterAverage %>%
  left_join(Tesremos_Locations, by = "ID")
```

Now we plot the average water deficit in March

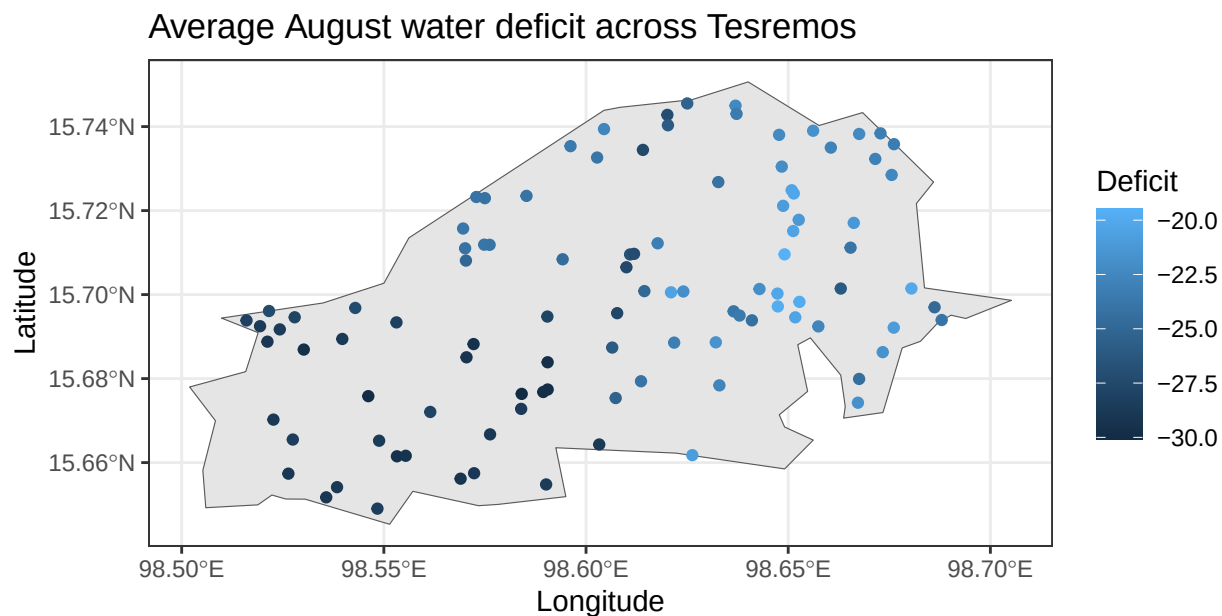
```
ggplot( TesremosShape ) + theme_bw() + geom_sf() +
  geom_point( data=MarchWater_Merged, aes( x=Longitude, y=Latitude, color = Deficit) ) +
  labs(title = "Average March water deficit across Tesremos")
```



The plot above tells us that in March the water deficit is on average between -14% and -24% across all 102 sites. We can see that there is deficit across Tesremos, with the greatest deficit in the North, with a deficit reaching more than 24% . The general pattern is that there is more deficit around the North and East borders, while the least deficit appears to be in the South-West and centre.

Now we plot the average water deficit in August.

```
ggplot( TesremosShape ) + theme_bw() + geom_sf() +
  geom_point( data=AugustWater_Merged, aes( x=Longitude, y=Latitude, color = Deficit) ) +
  labs(title = "Average August water deficit across Tesremos")
```



The plot above tells us that in August the water deficit is on average between -20% and -30% across all 102 sites. We can see that there is deficit across Tesremos, with the greatest deficit in the South-West of less than -30%, while the North-East has the least deficit of around -20%.

Next we explore dependence

To explore dependence on the monthly average deficit in groundwater levels for the 102 sites, it would be useful to use a semi-variogram. We first need to consider if our key assumptions are reasonable. We assume that the expectation of the water deficit is constant across Tesremos as we are working with deviations from historical averages at each site, suggesting the data is centred around 0. While the spatial plots suggest some regional differences, there is no clear, continuous trend across Tesremos. Therefore, assuming isotropy appears to be reasonable, although mild deviations cannot be entirely ruled out. Our next assumption is that the spatial dependence in the data is a function of spatial distance only. From the plots above we can see that water deficit appears to be similar in the same regions of Tesremos, hence we can use this assumption.

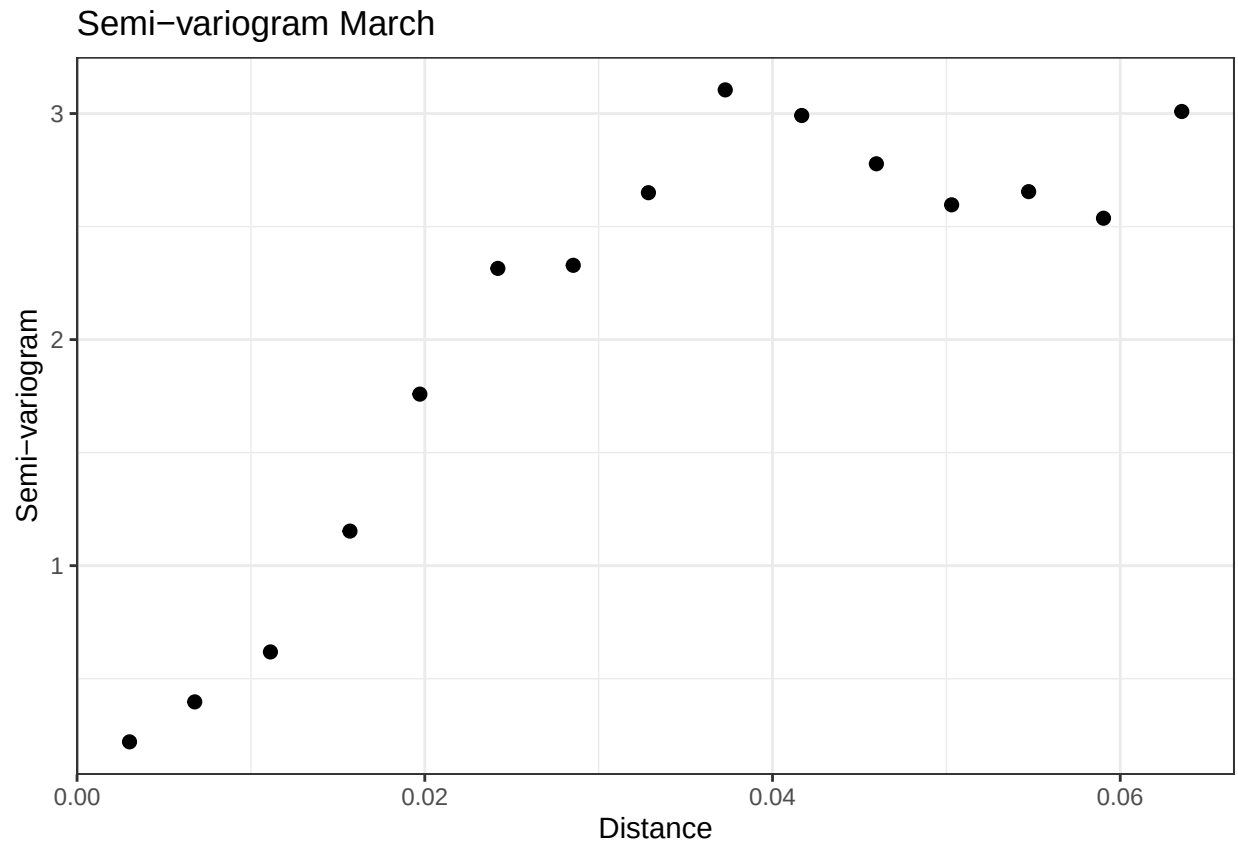
March Semi-variogram

First we need to ensure there are no missing values, and so we need to convert `Marchwater_Merged` back to a data frame, then remove any missing values before converting it back to a spatial points data frame.

```
MarchWater_Merged_df <- as.data.frame(MarchWater_Merged)
MarchWater_Merged_clean <- MarchWater_Merged_df %>%
  drop_na(Longitude, Latitude, Deficit)
coordinates(MarchWater_Merged_clean) <- ~Longitude + Latitude
```

We can now plot the semi-variogram for March

```
gamma_hat <- variogram(Deficit ~ Longitude + Latitude, data = MarchWater_Merged_clean)
ggplot(gamma_hat, aes(x = dist, y = gamma / 2)) + geom_point(size = 2) +
  labs(title = "Semi-variogram March") + theme_bw() +
  labs(x = "Distance", y = "Semi-variogram")
```



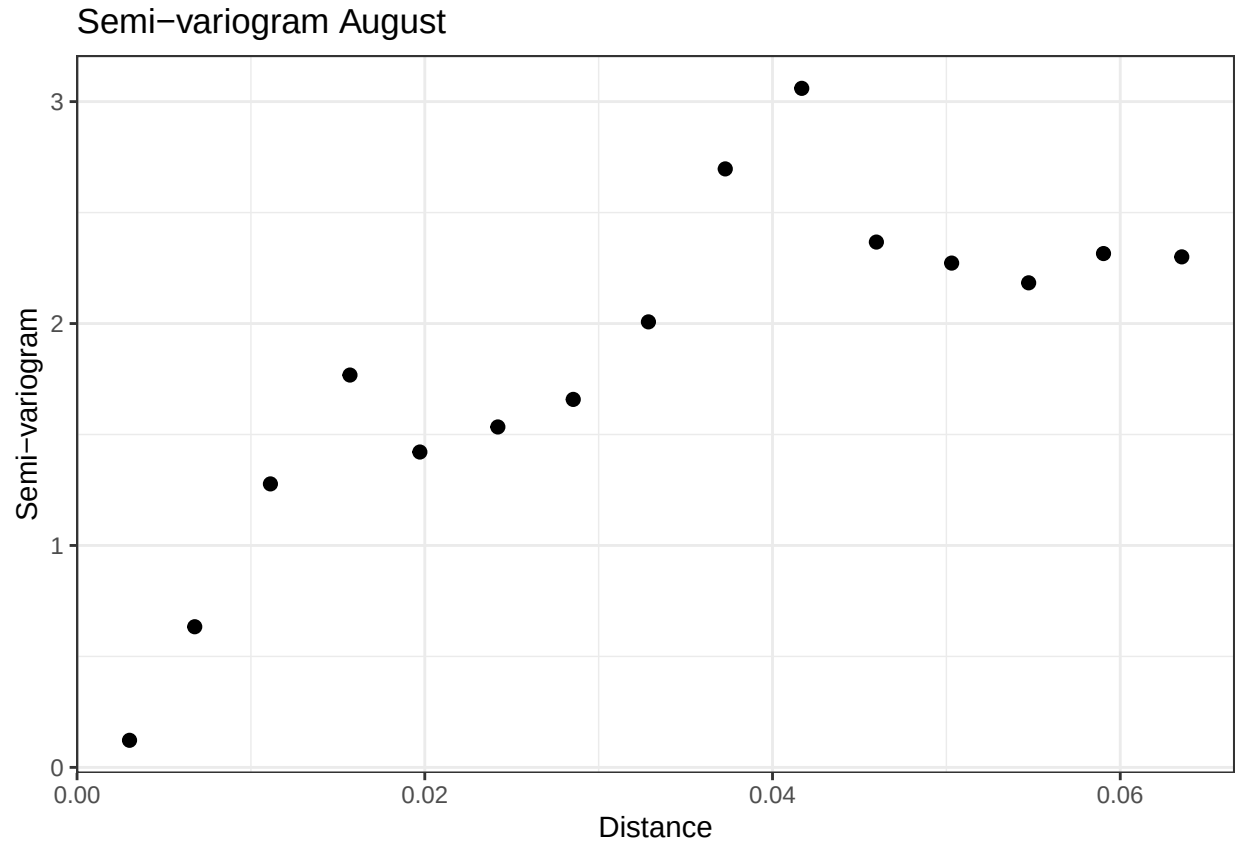
Between 0 and 0.035, we can see that dependence between pairs of sites decreased with increasing spatial distance, however after it appears to level off, suggesting that beyond this distance there is no significant spatial dependence, indicating independence at large spatial distance.

We repeat the same to find the August Semi-variogram

```
AugustWater_Merged_df <- as.data.frame(AugustWater_Merged)
AugustWater_Merged_clean <- AugustWater_Merged_df %>%
  drop_na(Longitude, Latitude, Deficit)
coordinates(AugustWater_Merged_clean) <- ~Longitude + Latitude
```

We can now plot the semi-variogram

```
gamma_hat <- variogram(Deficit ~ Longitude + Latitude, data = AugustWater_Merged_clean)
ggplot(gamma_hat, aes(x = dist, y = gamma / 2)) + geom_point(size = 2) +
  labs(title = "Semi-variogram August") + theme_bw() +
  labs(x = "Distance", y = "Semi-variogram")
```



Initially, as distance increases so does semi-variogram, indicating decreasing dependence, then it falls between 0.015 and 0.02, indicating a range of spatial dependence at this distance, before increasing again, until it appears to level off beyond 0.04, indicating that there is no longer any spatial dependence.

- ii. *Produce maps which provide estimates for the monthly average deficit in groundwater levels across the region for March and August 2024. Discuss the reliability of your estimates.* ## Now we use spatial interpolation to provide estimates for the monthly average deficit Inverse Distance Weighting function from lecture notes

```
IDW <- function( X, S, s_star, p){
  d <- sqrt( (S[,1]-s_star[1])^2 + (S[,2]-s_star[2])^2 )
  w <- d^(-p)
  if( min(d) > 0 )
    return( sum( X * w ) / sum( w ) )
  else
    return( X[d==0] )
}
```

We define a grid that covers the study space

```
points_lon <- seq(98.5, 98.75, by = 0.001)
points_lat <- seq(15.64, 15.76, by = 0.001)
pixels <- as.matrix(expand.grid(Lon = points_lon, Lat = points_lat))
```

We create coordinates for the March data

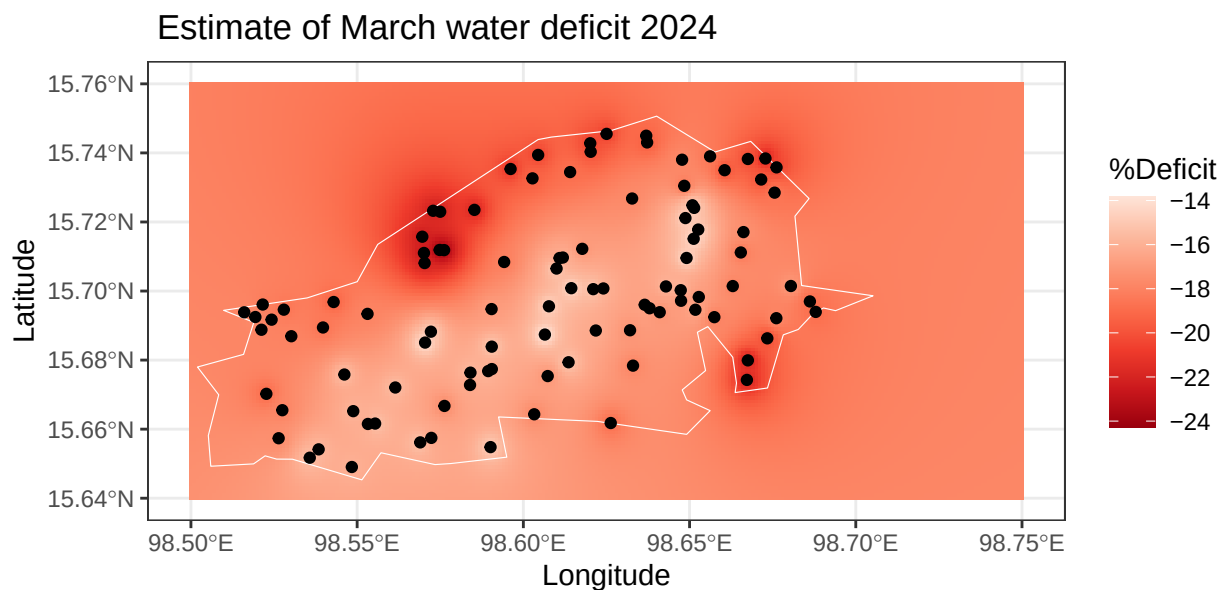
```
coord <- cbind(MarchWater_Merged$Longitude, MarchWater_Merged$Latitude)
```

We then apply IDW for $p = 2$ directly on the grid points and create the prediction frame

```
p2 <- apply(pixels, 1, function(p) {
  IDW(X = MarchWater_Merged$Deficit, S = coord, s_star = p, p = 2)})
Predict <- data.frame(Lon = pixels[, 1], Lat = pixels[, 2], value = p2)
```

We then plot the results

```
ggplot(Predict) + theme_bw() +
  geom_raster(aes(x = Lon, y = Lat, fill = value)) +
  scale_fill_distiller(palette = "Reds") +
  geom_sf(data = TesremosShape, alpha = 0.0, color = "white") +
  geom_point(data = MarchWater_Merged, aes(x = Longitude, y = Latitude)) +
  labs(title = "Estimate of March water deficit 2024", fill = "%Deficit",
       x = "Longitude", y = "Latitude")
```



This plot estimates that the water deficit in March is mostly between -14% and -24%, with the least deficit in the centre of the region, along a North East running line, and most deficit running along the boarder in the North and East, where there are 3 clusters of large water deficit. especially in the North of the country.

We create coordinates for the August data

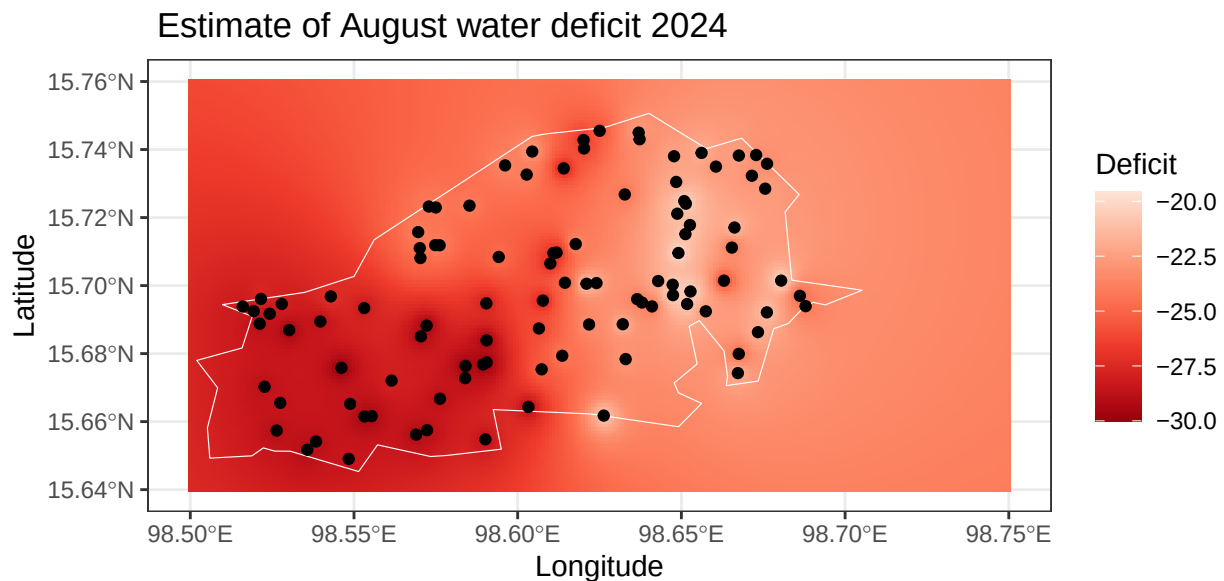
```
coord2 <- cbind(AugustWater_Merged$Longitude, AugustWater_Merged$Latitude)
```

We then apply IDW for $p = 2$ directly on the grid points and create the prediction frame

```
p22 <- apply(pixels, 1, function(p) {
  IDW(X = AugustWater_Merged$Deficit, S = coord2, s_star = p, p = 2)})
Predict2 <- data.frame(Lon = pixels[, 1], Lat = pixels[, 2], value = p22)
```

We then plot the results

```
ggplot(Predict2) + theme_bw() +
  geom_raster(aes(x = Lon, y = Lat, fill = value)) +
  scale_fill_distiller(palette = "Reds") +
  geom_sf(data = TesremosShape, alpha = 0.0, color = "white") +
  geom_point(data = AugustWater_Merged, aes(x = Longitude, y = Latitude)) +
  labs(title = "Estimate of August water deficit 2024", fill = "Deficit",
       x = "Longitude", y = "Latitude")
```



This plot estimates August water deficit to mostly be between -20% and -30% across the region. It estimates the least deficit in the South-West of the region, and most in the North-East of the region.

In our Analysis, we have assumed that groundwater levels vary widely over short distance, which in the real world may not be true. We do not know if the sites are evenly spread out across the region, but given that there are 102 sites across the area, we can expect estimates to be fairly reliable. We have chosen a power parameter of 2, while a lower parameter smooths noise, it risks not presenting extremes, while a large p risks over fitting noise.

Question 3

The Isles of Sofara are one of the most beautiful spots across Utopia and very popular with tourists. There are a total of five islands: Calmorra, Ecliptria, Justitia, Paxora and Solvenya. You have been provided with a shapefile of the islands; to hide their location, constants have been added to the latitude and longitude coordinates, but the shapes they define are correct.

Over recent decades, the islands have been frequently hit by devastating tropical storms and the government of Utopia was repeatedly blamed for their poor disaster response. To address this issue, the government of Utopia set up a helpline in 2020 - every person in distress due to a tropical storm should send a text message to the number 971 with the hashtag #IoSTSHelp and, if possible, provide their location information. Since 2020, a few tropical storms hit the islands and, while the introduction of the helpline and hashtag was generally successful, the current setup requires a government employee to manually evaluate the text messages received, which leads to delays in the disaster response.

The government of Utopia has now approached you for help with the analysis of the text messages sent to 971. Specifically, for the messages with the hashtag #IoSTSHelp, they ask you to:

- i. *Visualize the number of messages per island and identify, as precisely as possible, the areas from which the most distress messages were sent.*

We first read the files

```
Sofara_Messages <- read.csv("Isles_of_Sofara_Messages.csv")
SofaraShape <- read_sf("Isles_of_Sofara.shp")
```

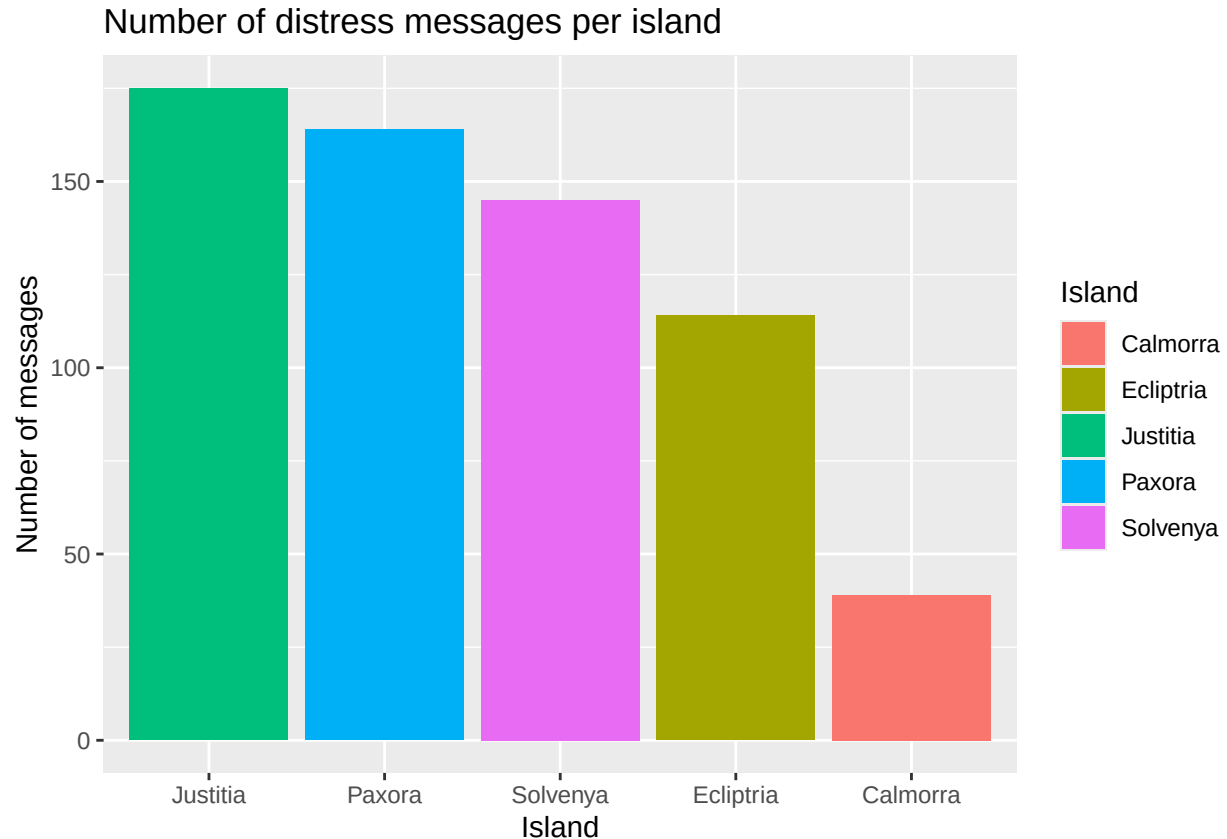
We then select the distress messages and count how many there are per island

```
Distress <- Sofara_Messages %>%
  filter(str_detect(Message, "#IoSTSHelp"))

Distress_counts <- Distress %>%
  count(Island, name = "Distress_Count")
```

We can use a bar plot to visualise how many distress messages there are per island

```
ggplot(Distress_counts, aes(x = reorder(Island, -Distress_Count),
                             y = Distress_Count, fill = Island)) +
  geom_bar(stat = "identity") +
  labs(title = "Number of distress messages per island",
       x = "Island", y = "Number of messages")
```



From the plot above, we can see that there is a range of number of distress messages per island, with the most messages coming from ‘Justitia’, with roughly 175 messages and the least from ‘Calmorra’ with around 35 messages. It is clear that the islands: ‘Justitia, Paxora and Solvenya’ sent the most messages. From this plot alone we cannot visualise where on each island the most distress messages were coming from, and so it would be useful to visualise the locations of the messages on each island.

Next we want to visualise the locations of the distress messages in each island.

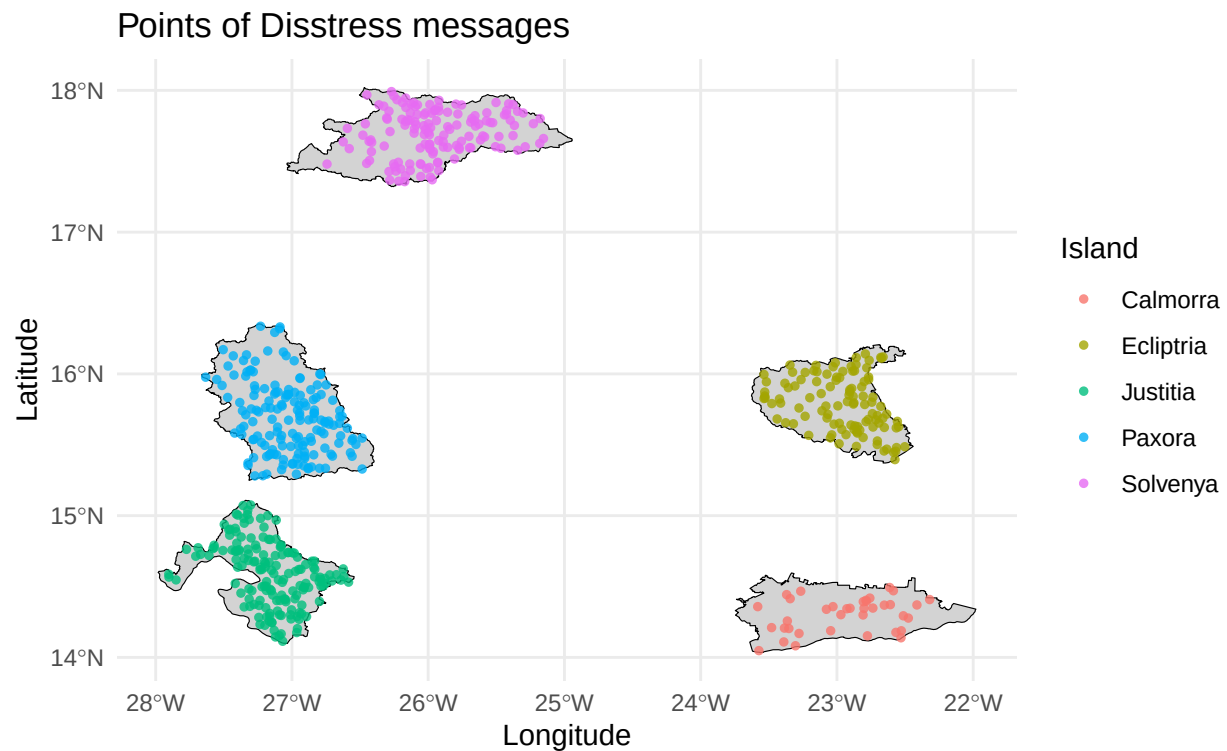
We now join the distress counts with the shapefile and convert the distress data into a shapefile, removing any data that has no location.

```
SofaraShape <- SofaraShape %>%
  rename(Island = Name) %>% # Make sure shapefile column matches
  left_join(Distress_counts, by = "Island")

Distress_geo <- Distress %>%
  filter(!is.na(Longitude) & !is.na(Latitude)) %>%
  st_as_sf(coords = c("Longitude", "Latitude"), crs = 4326)
```

We can plot the distressed messages on each island now.

```
ggplot() + geom_sf(data = SofaraShape, fill = "lightgrey", color = "black") +
  geom_sf(data = Distress_geo, aes(color = Island), alpha = 0.8, size = 1) +
  labs(title = "Points of Distress messages", x = "Longitude", y = "Latitude") +
  theme_minimal()
```



This plot tells us that many distress messages are coming from across the islands, with a lot of messages coming from the East of Solvenya. This plot lacks to identify messages relative to area, and where the most messages are coming from, and so a PPP density plot would be helpful.

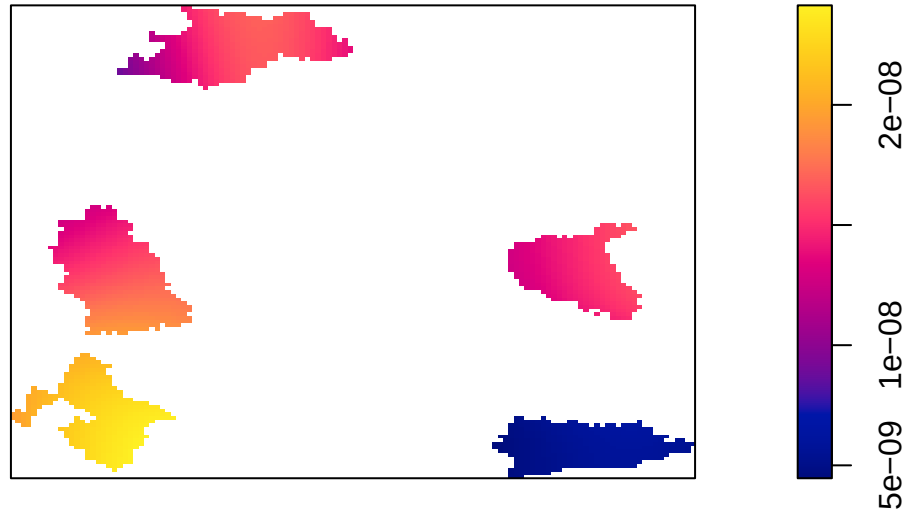
We first change coordinate systems

```
I <- SofaraShape %>%
  st_transform(crs = 3857)
Distress_transformed <- Distress_geo %>%
  st_transform(crs = 3857)
```

Then we create the ppp object, and plot the correct smooth kernel intensity.

```
I_owin <- as.owin(I)
coords <- st_coordinates(Distress_transformed)
msgs_ppp <- ppp(x = coords[,1], y = coords[,2], window = I_owin)
lambdaC_msgs <- density.ppp(msgs_ppp, edge = TRUE)
PPP_plot <- plot(lambdaC_msgs, main = "Distress Message Density - Isles of Sofara")
```

Distress Message Density – Isles of Sofara



From the plot above we can conclude that the highest density of distress messages came from Justitia (across the whole island, especially in the South). The island Paxora also had a high intensity of distress messages sent, especially in the South of the island. This suggests that in 2020, the storms most frequently hit Justitia, and even the South of Paxora. Generally speaking, we can see a similar intensity in distressed messages from Paxora, Solvany and Ecliptria, implying that tropical storms hit these islands with similar occurrence. The least distressed messages came from Calmorra, indicating there may be less frequent storms there. It should be noted however, that we do not have access to information regarding the islands access to phones and electricity, which could in turn affect the number of distressed messages received, and in turn how we interpret where the most frequent tropical storms are.

- ii. *Develop an approach to identify the kind of help required. The four key categories are (a) lack of food, (b) lack of fuel, (c) lack of shelter and (d) need for medical assistance. Use your approach to provide estimates for the number of people who required help related to the categories (a)-(d).*

The government of Utopia is planning to use your analysis approach for any future tropical storms. Specifically, they want to use your approach to identify the areas that need help and the kind of help required, without a government employee having to manually evaluate the text messages. Discuss the strengths and limitations of your analysis approach regarding this aspect.

Answer

It would first be useful to see if the demand for help in different categories changes for each island. We do our best to group distressed messages into the categories: lack of food, lack of fuel, lack of shelter and need for medical assistance.

```
searchwords <- list(
  food = c("food", "eat", "hungry", "meal"),
  fuel = c("fuel", "generator", "cooking", "heating", "gas"),
  shelter = c("shelter", "roof", "homeless", "home", "house"),
  medical = c("medical", "injured", "medicine", "wounds"))
```

We ensure the message are all in lower case

```
Distress$Message_lower <- tolower(Distress$Message)
```

We search for the key words to separate the messages into the 4 categories

```
Distress_types <- Distress %>%
  mutate( Food_help = grepl(paste(searchwords$food, collapse = "|"), Message_lower),
    Fuel_help = grepl(paste(searchwords$fuel, collapse = "|"), Message_lower),
    Shelter_help = grepl(paste(searchwords$shelter, collapse = "|"), Message_lower),
    Medical_help = grepl(paste(searchwords$medical, collapse = "|"), Message_lower))
```

We then count the number of messages in each category type

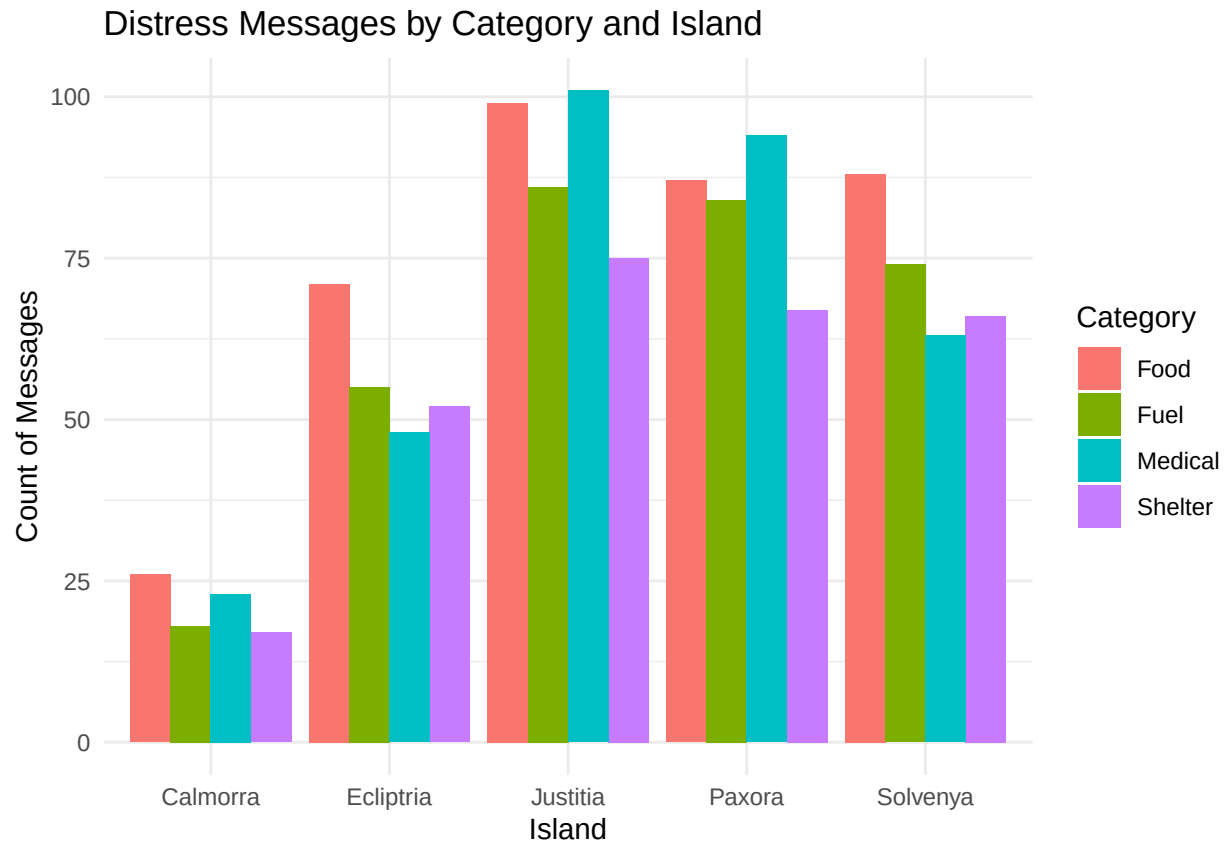
```
Island_category_counts <- Distress_types %>%
  group_by(Island) %>%
  summarise(Food = sum(Food_help), Fuel = sum(Fuel_help), Shelter = sum(Shelter_help),
    Medical = sum(Medical_help))
```

In order to produce a bar plot, it would be useful to change the format of the data

```
Island_category_long <- Island_category_counts %>%
  pivot_longer(cols = -Island, names_to = "Category", values_to = "Count")
```

The data is now ready to plot

```
ggplot(Island_category_long, aes(x = Island, y = Count, fill = Category)) +
  geom_bar(stat = "identity", position = "dodge") + labs(
    title = "Distress Messages by Category and Island", x = "Island",
    y = "Count of Messages") + theme_minimal()
```



From the plot above we can see that lack of food was the most prevalent distress messages for Calmorra, Ecliptria and Solvenya. For Justitia and Paxora, medical attention was the most common reason for distress messages. This makes sense, seeing that our analysis above showed us that most storms hit these 2 islands, so more people are likely to be injured. We can see that there tends to be variation in requirement for medical attention and lack of shelter compared to the other 2 categories within each island, with medical attention being requested the least in Ecliptria and Solvenya, while lack of shelter was messaged the least in Calmorra, Justitia and Paxora. Perhaps there is an economic reason for this, however we would need the GDP of each island to further explore this.

We now look at the overall requirements for the 5 islands together.

```
Full_counts <- Island_category_long %>%
  group_by(Category)%>%
  summarise(Total = sum(Count))
Full_counts
```

```
## # A tibble: 4 x 2
##   Category Total
##   <chr>      <int>
## 1 Food        371
## 2 Fuel        317
## 3 Medical     329
## 4 Shelter     277
```

Based on our estimates, majority of people needed more food (around 371). We can see that the estimates for Fuel (317) and Medical(329) help were very similar, and seeing that this is an estimate they may need to be treated with equal urgency. We can see that the lowest estimate was for lack of shelter (277).

The strengths of this approach is that from the analysis of where the most distressing messages come from, we can see which parts of the Isle of Sofara need the most attention in terms of keeping the people safe from future tropical storms. Using key words limits the need for someone to manually read each text, making it quick and easy to interpret the data. Furthermore if new words emerge, they can be easily added to the key. The limitations of the approach is that by creating a key word list, synonyms for the words will not be picked up on, and so we cannot fully trust our estimates for the number of distressed messages belonging to each category. Furthermore, if one message asked for help in more than one category, this approach will only categorise it in one.